实验四 存储器阵列设计

一、实验目的

1 掌握 Verilog 语言和 Vivado、Logisim 开发平台的使用；

2 掌握存储器和寄存器组的设计和测试方法。

二、实验内容

1 存储器设计与测试

2 寄存器组设计与测试

三、实验要求

1 掌握 Vivado 或 Logisim 开发工具的使用，掌握以上电路的设计和测试方法；

2 记录设计和调试过程（Verilog 代码/电路图/表达式/真值表，Vivado 仿真结果，Logisim 验证结果等）；

3 分析 Vivado 仿真波形/Logism 验证结果，注重输入输出之间的对应关系。

四、实验过程及分析

（一）1K×16bit 存储器 双数据线

1、代码：

```
module RAM_1Kx16(Data_out, Addr, Rst, RE, WE, CS, CLK, Data_in);
    parameter Addr_Width = 10;  //参数化地址线宽
    parameter Data_Width = 16;  //参数化数据线宽
    parameter SIZE = 2 ** Addr_Width;  //参数化大小 1024
    output reg [Data_Width-1:0] Data_out;//数据输出
    input [Addr_Width-1:0] Addr;         //地址数据
    input Rst; //复位信号
    input RE;  //读使能信号
    input WE;  //写使能信号
    input CS;  //使能/片选信号
    input CLK; //时钟信号
    input [Data_Width-1:0] Data_in;  //数据输入

    integer i;//临时变量，用于 for 循环
    reg [Data_Width-1:0] RAM [SIZE-1:0]; //1024*16 的 RAM

    //always @(posedge CLK) begin //时钟同步
    always @(*) begin  //异步
        casex({CS, Rst, RE, WE})
            4'bx1xx : for(i = 0;i <= SIZE-1;i = i+1) RAM[i] = 0;
```

```
            4'b1010 : Data_out <= RAM[Addr];  //读数据
            4'b1001 : RAM[Addr] <= Data_in;   //写数据
            default : Data_out = 16'bz;
        endcase
    end
endmodule
```

分析：RAM_1Kx16 模块采用参数化的地址宽度 (Addr_Width=10) 和数据宽度 (Data_Width=16)，内部使用大小为 1024 的寄存器数组 (RAM[0:1023]) 来存储 16 位数据。该存储器具有独立的数据输入 (Data_in) 和数据输出 (Data_out) 总线，因此读写不共用同一数据线。模块通过组合逻辑的 always 块实现功能：根据片选 CS、复位 Rst、读使能 RE 和写使能 WE 信号的组合决定操作。当复位 Rst 信号为高时，无论 CS 状态如何，执行清零操作，将所有存储单元重置为 0。若未复位且片选 CS 有效 (为 1)，模块区分读写模式：当 WE＝1 且 RE＝0 时执行写操作，将 Data_in 输入数据写入到选定的地址 Addr 对应的存储单元；当 RE＝1 且 WE＝0 时执行读操作，从 RAM 数组中取出 Addr 地址存储的数据，经由 Data_out 输出。为了避免无效输出干扰总线，当芯片未选中 (CS＝0) 或无读写操作时，Data_out 被驱动为高阻态 (16'bz)，释放数据总线。整个设计未使用时钟触发（采用组合电路实现异步存取），因此读写在信号满足条件时便立即发生，实现对存储器的异步访问。

2、仿真文件：

```
module sim_RAM_1Kx16;
    parameter CLK_PERIOD = 10;
    parameter ADDR_WIDTH = 10;
    parameter DATA_WIDTH = 16;
    parameter SIZE = 2 ** ADDR_WIDTH;

    reg  [ADDR_WIDTH-1:0] Addr;
    reg                   Rst;
    reg                   RE;
    reg                   WE;
    reg                   CS;
    reg                   CLK;
    reg  [DATA_WIDTH-1:0] Data_in;
    wire [DATA_WIDTH-1:0] Data_out;
    integer i;
    RAM_1Kx16 uut (
        .Data_out(Data_out),
        .Addr     (Addr),
```

```
    .Rst     (Rst),
    .RE      (RE),
    .WE      (WE),
    .CS      (CS),
    .CLK     (CLK),
    .Data_in  (Data_in)
);

// 时钟生成
initial begin
    CLK = 0;
    forever #(CLK_PERIOD/2) CLK = ~CLK;
end

initial begin
    Addr     = 0;
    Rst      = 0;
    RE       = 0;
    WE       = 0;
    CS       = 0;
    Data_in  = 0;
    #20;

    // 测试 1: 复位功能 (清零所有存储单元)
    Rst = 1;
    #15;  // 保持 15ns
    Rst = 0;
    #20;

    // 测试 2: 写操作 (向几个地址写入测试数据)
    CS = 1;
    WE = 1; RE = 0;

    Addr = 10'h0;   Data_in = 16'hAAAA; #10;
    Addr = 10'h3FF; Data_in = 16'h5555; #10;  // 写入最后一个地址
    Addr = 10'h123; Data_in = 16'h1234; #10;
    Addr = 10'h0;   Data_in = 16'h1111; #10;  // 覆盖地址 0
```

```
        // 测试 3: 读操作
        WE = 0; RE = 1;

        Addr = 10'h0;   #10;   // 应该读到 16'h1111
        Addr = 10'h123; #10;   // 应该读到 16'h1234
        Addr = 10'h3FF; #10;   // 应该读到 16'h5555

        // 测试 4: CS 信号有效性 (CS=0 时读写应无效)
        CS = 0;  // 禁用芯片
        Addr = 10'h0; Data_in = 16'hDEAD; WE = 1; RE = 0; #10;  // 写操作
应无效
        WE = 0; RE = 1; #10;  // 读操作应无效，应输出高阻

        // 测试 5: 同时读写(RE=WE=1)的情况
        CS = 1;
        Addr = 10'h0; Data_in = 16'hBEEF; RE = 1; WE = 1; #10;

        // 测试 6: 模拟读取文件
        Rst = 1; #15; Rst = 0;  // 先复位

        uut.RAM[0]  <= 16'hAf45;
        uut.RAM[1]  <= 16'h6781;
        uut.RAM[2]  <= 16'h901C;
        uut.RAM[3]  <= 16'h3456;
        uut.RAM[4]  <= 16'h789F;
        uut.RAM[5]  <= 16'h1234;
        uut.RAM[6]  <= 16'h567D;
        uut.RAM[7]  <= 16'h9012;
        uut.RAM[8]  <= 16'h34E6;
        uut.RAM[9]  <= 16'h7890;
        uut.RAM[10] <= 16'h1234;
        uut.RAM[11] <= 16'h5678;

        #10;  // 等待稳定

        // 读取刚载入的数据
        WE = 0; RE = 1;
```

```
        for (i = 0; i < 16; i = i + 1) begin
            Addr = i; #10;
        end

        // 结束仿真
        #50;
    end
endmodule
```

仿真策略：本模块的测试分多个情景逐一验证其功能：

测试 1：复位功能验证。将 Rst 信号置 1 并保持一段时间后解除，模拟整体清零操作；复位完成后，存储阵列中的所有数据应被清零（后续读操作将返回 0）。

测试 2：写操作测试。在 CS＝1 使能芯片、WE＝1 的条件下，分别向多个地址连续写入数据：如地址 0 写入 16’hAAAA，地址 0x3FF（1023）写入 16’h5555（最高地址），地址 0x123 写入 16’h1234，最后再次对地址 0 写入 16’h1111 覆盖原数据。通过这些操作验证写使能的有效性，且对同一地址后写入的数据会覆盖先前值。

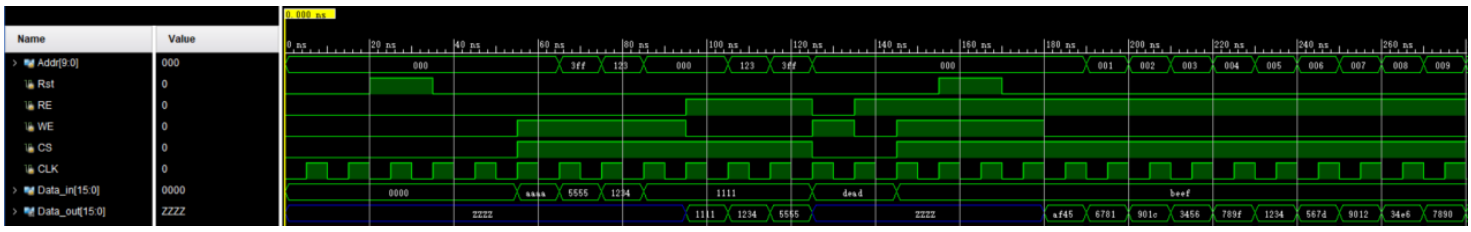
测试 3：读操作测试。将 WE 置 0、RE 置 1 后，对之前写入过的地址执行读操作，例如依次读取地址 0、0x123、0x3FF。预期 Data_out 分别输出 16’h1111、16’h1234、16’h5555，与先前写入的数据一致，从而验证读功能正确。

测试 4：片选信号 CS 测试。当 CS＝0 禁用芯片时，无论 WE 或 RE 如何设置，读写操作都应无效。仿真中在 CS 拉低的情况下尝试写地址 0 数据 16’hDEAD，然后进行读操作。由于芯片未选中，实际存储内容不会改变，读端口输出应保持高阻态，验证 CS 控制的有效性。

测试 5：同时读写测试。当 RE 和 WE 同时为 1（不合法的操作组合）时，观察模块行为。预期模块不会执行实际读或写操作，此时 Data_out 保持高阻态以避免总线冲突，体现对非法操作组合的保护。

测试 6：预加载数据读取测试。先通过 Rst 清零存储器，然后在仿真中直接预先填充 uut 的 RAM 数组若干地址为特定数据，模拟从文件载入内容。随后将 WE＝0、RE＝1，循序增大 Addr 从 0 读到 15。观察 Data_out 输出应依次返回预先写入的值，验证存储器能够正确保存并输出预装载的数据。

3、SIMULATION：



分析：时钟 CLK 从 0ns 开始以 10ns 的周期振荡。仿真启动时所有控制信号和数据

线均为 0，CS 也拉低禁止芯片。20ns 时产生一次复位脉冲 (Rst=1)，持续到 35ns，期间内部 1K×16 RAM 的所有存储单元被清零。复位结束后，在 55ns 开始写操作测试：先将 CS 置 1 使能芯片，设置 WE=1、RE=0 进入写模式。在 55ns 时，Addr 指向 0x000 地址，Data_in 输入 0xAAAA，该数据被立即写入地址 0 单元。65ns 时，Addr 切换为 0x3FF，Data_in 提供 0x5555 并写入此单元；75ns 时写入地址 0x123 数据 0x1234；85ns 时再次访问地址 0x000 写入新数据 0x1111（覆盖先前写入的 0xAAAA）。接下来在 95ns 切换为读模式 (WE=0, RE=1)，依次读取上述写入的数据：95ns 读地址 0x000，观察到 Data_out 输出变为 0x1111；105ns 时地址改为 0x123，Data_out 输出 0x1234；115ns 时地址 0x3FF，Data_out 正确输出 0x5555。125ns 开始验证 CS 信号作用：将 CS 拉低为 0 并尝试写操作——Addr=0x000，Data_in=0xDEAD，WE=1；由于芯片被禁用，此时未发生实际写入，Data_out 也因无读使能而保持高阻态 Z。随后在 135ns 尝试在 CS=0 下读出，设 RE=1、WE=0，但 Data_out 仍为高阻态，没有输出有效数据，证明 CS=0 时读写均无效。145ns 进行 RE 和 WE 同时为 1 的冲突测试：Addr=0x000，Data_in=0xBEEF 且 CS=1，但由于读写冲突电路不执行有效操作，此时 Data_out 同样呈高阻状态。最后在 155ns 再次拉高 Rst 复位 15ns 清零存储器，随后通过测试平台直接预加载了若干数据到存储单元。180ns 起系统连续读取地址 0x000 到 0x00F：从 180ns 开始 Addr 每隔 10ns 加 1，Data_out 将依次输出预存的数据。其中地址 0 输出 0xAF45，地址 1 输出 0x6781，...，地址 7 输出 0x9012，地址 8 输出 0x34E6，一直到地址 11 输出 0x5678；对于未预加载的地址 (例如 12~15)，300ns 以后 Data_out 则返回 0。整个仿真过程中，各次读写操作的时序关系清晰可见，验证了 RAM_1Kx16 模块的读写控制和数据输出功能的正确性。

（二）4K×32 位存储器

1、代码：

（1）2-4 译码器：

```
module Decoder24(
    output reg [3:0] CS_i,
    input  [1:0] Sel
);
    always @(*) begin
        case(Sel)
            2'b00: CS_i = 4'b0001;
            2'b01: CS_i = 4'b0010;
            2'b10: CS_i = 4'b0100;
            2'b11: CS_i = 4'b1000;
        endcase
    end
endmodule
```

（2）主模块：

```
module RAM_4Kx32(
    output [31:0] Data_out,      // 32 位数据输出
    input [11:0] Addr,           // 12 位地址线
    input Rst,                  // 复位信号
    input RE,                   // 读使能
    input WE,                   // 写使能
    input CS,                   //片选
    input CLK,                  // 时钟
    input [31:0] Data_in         // 32 位数据输入
);

    parameter Addr_Width = 12;   // 参数化地址线宽
    parameter Data_Width = 32;   // 参数化数据线宽

    wire [3:0] CS_i;             // 4 位片选信号组

    // 2-4 译码器：用地址最高 2 位生成 4 个片选信号
    Decoder24 decoder_inst(CS_i, Addr[Addr_Width-1:Addr_Width-2]);

    // 4 个 RAM_1Kx16 模块，每个分为高 16 位和低 16 位两部分
    // CS0 组：地址范围 0x000-0x3FF
    RAM_1Kx16 CS0_H_16bit(
        .Data_out(Data_out[31:16]),
        .Addr(Addr[9:0]),
        .Rst(Rst), .RE(RE), .WE(WE), .CS(CS_i[0]), .CLK(CLK),
        .Data_in(Data_in[31:16])
    );
    RAM_1Kx16 CS0_L_16bit(
        .Data_out(Data_out[15:0]),
        .Addr(Addr[9:0]),
        .Rst(Rst), .RE(RE), .WE(WE), .CS(CS_i[0]), .CLK(CLK),
        .Data_in(Data_in[15:0])
    );

    // CS1 组：地址范围 0x400-0x7FF
    RAM_1Kx16 CS1_H_16bit(
        .Data_out(Data_out[31:16]),
```

```
    .Addr(Addr[9:0]),
    .Rst(Rst), .RE(RE), .WE(WE), .CS(CS_i[1]), .CLK(CLK),
    .Data_in(Data_in[31:16])
);
RAM_1Kx16 CS1_L_16bit(
    .Data_out(Data_out[15:0]),
    .Addr(Addr[9:0]),
    .Rst(Rst), .RE(RE), .WE(WE), .CS(CS_i[1]), .CLK(CLK),
    .Data_in(Data_in[15:0])
);

// CS2 组：地址范围 0x800-0xBFF
RAM_1Kx16 CS2_H_16bit(
    .Data_out(Data_out[31:16]),
    .Addr(Addr[9:0]),
    .Rst(Rst), .RE(RE), .WE(WE), .CS(CS_i[2]), .CLK(CLK),
    .Data_in(Data_in[31:16])
);
RAM_1Kx16 CS2_L_16bit(
    .Data_out(Data_out[15:0]),
    .Addr(Addr[9:0]),
    .Rst(Rst), .RE(RE), .WE(WE), .CS(CS_i[2]), .CLK(CLK),
    .Data_in(Data_in[15:0])
);

// CS3 组：地址范围 0xC00-0xFFF
RAM_1Kx16 CS3_H_16bit(
    .Data_out(Data_out[31:16]),
    .Addr(Addr[9:0]),
    .Rst(Rst), .RE(RE), .WE(WE), .CS(CS_i[3]), .CLK(CLK),
    .Data_in(Data_in[31:16])
);
RAM_1Kx16 CS3_L_16bit(
    .Data_out(Data_out[15:0]),
    .Addr(Addr[9:0]),
    .Rst(Rst), .RE(RE), .WE(WE), .CS(CS_i[3]), .CLK(CLK),
    .Data_in(Data_in[15:0])
);
```

endmodule

分析：RAM_4Kx32 模块实现了 4K × 32 位的数据存储，其设计采用分层结构，由多个较小存储单元组合构成。模块含有一个 2-4 译码器 Decoder24，用于将 12 位地址 Addr 的高 2 位进行译码，产生 4 个片选信号 CS_i[3:0]，分别对应 4 个 1K 大小的存储块。地址空间因此被均分为 4 段：当 Addr[11:10]＝00 时选中 CS0 块（地址范围 0x000~0x3FF），01 对应 CS1 块（0x400~0x7FF），10 对应 CS2 块（0x800~0xBFF），11 对应 CS3 块（0xC00~0xFFF）。模块内部实例化了 4 组 RAM_1Kx16 子模块，每组包括两片 1K×16bit 存储器，分别存储 32 位数据的高 16 位和低 16 位。对于每个地址块，当对应的 CS_i 信号为 1 时，该块的两片存储器同时以 Addr 的低 10 位作为地址接入，并共享上层传来的 Rst、RE、WE 控制信号和 CLK 时钟，从而对 32 位数据的两部分进行同步的读写操作；未被选中的块其 CS 信号为 0，不参与读写。Data_in 和 Data_out 总线按照高 16 位和低 16 位拆分后分别连接到各组子存储器，最终在顶层组合为完整的 32 位输入/输出。通过这种“位扩展 + 字扩展”的设计（纵向通过片选译码扩展存储深度，横向并联芯片扩展数据位宽），RAM_4Kx32 将多个 1K×16 模块有机组成一个 4K×32 的存储器。模块还设有一个全局片选输入 CS，当 CS＝0 时可禁止整个存储器阵列响应读写操作，只有当 CS＝1 且地址译码选定相应块时，读写过程才会发生。顶层模块据此协调各子模块的片选与总线输出：每次仅允许选中的存储块驱动 Data_out，总线上的其它分支保持高阻，从而保证数据正确传递且避免冲突。

2、仿真文件：

```
module sim_RAM_4Kx32;
    parameter CLK_PERIOD = 10;
    reg  [11:0] Addr;
    reg         Rst;
    reg         RE;
    reg         WE;
    reg         CS;
    reg         CLK;
    reg  [31:0] Data_in;
    wire [31:0] Data_out;

    RAM_4Kx32 uut (
        .Data_out(Data_out),
        .Addr    (Addr),
        .Rst     (Rst),
        .RE      (RE),
        .WE      (WE),
```

```
    .CS     (CS),
    .CLK    (CLK),
    .Data_in (Data_in)
);

initial begin
    CLK = 0;
    forever #(CLK_PERIOD/2) CLK = ~CLK;
end

task test_write_read;
    input [11:0] test_addr;
    input [31:0] test_data;
    begin
        // 写操作
        CS = 1; WE = 1; RE = 0;
        Addr = test_addr; Data_in = test_data; #10;

        // 读操作
        WE = 0; RE = 1;
        Addr = test_addr; #10;
    end
endtask

// 主测试流程
initial begin
    // ===== 初始化所有信号 =====
    Addr = 0; Rst = 0; RE = 0; WE = 0; CS = 0; Data_in = 0;

    // ===== Test 1: 复位功能测试 =====
    Rst = 1; #15; Rst = 0; #10;

    // ===== Test 2: 4 个 RAM 块独立测试 =====
    // CS0 块测试 (地址范围: 0x000-0x3FF)
    test_write_read(12'h000, 32'hAAAA5555);
    test_write_read(12'h001, 32'h11223344);
    test_write_read(12'h3FF, 32'hDEADBEEF);
```

```
// CS1 块测试 (地址范围: 0x400-0x7FF)
test_write_read(12'h400, 32'hBBBB6666);
test_write_read(12'h555, 32'hFEDCBA98);
test_write_read(12'h7FF, 32'hCAFEBABE);

// CS2 块测试 (地址范围: 0x800-0xBFF)
test_write_read(12'h800, 32'hCCCC7777);
test_write_read(12'h9AA, 32'h87654321);
test_write_read(12'hBFF, 32'hFEEDFACE);

// CS3 块测试 (地址范围: 0xC00-0xFFF)
test_write_read(12'hC00, 32'hDDDD8888);
test_write_read(12'hD23, 32'h01234567);
test_write_read(12'hFFF, 32'hDEC0DED0);

// ===== Test 3: 跨块连续读写 =====
CS = 1; WE = 1; RE = 0;
Addr = 12'h3FF; Data_in = 32'h11111111; #10;  // CS0 最后一个地址
Addr = 12'h400; Data_in = 32'h22222222; #10;  // CS1 第一个地址
Addr = 12'h7FF; Data_in = 32'h33333333; #10;  // CS1 最后一个地址
Addr = 12'h800; Data_in = 32'h44444444; #10;  // CS2 第一个地址

// 读回验证
WE = 0; RE = 1;
Addr = 12'h3FF; #10;
Addr = 12'h400; #10;
Addr = 12'h7FF; #10;
Addr = 12'h800; #10;

// ===== Test 4: 片选信号验证 =====
// 写入 CS2 块
CS = 1; WE = 1; RE = 0;
Addr = 12'h900; Data_in = 32'h55555555; #10;

// 禁用 CS 后写入（应无效）
CS = 0;
Addr = 12'h900; Data_in = 32'hBADBADBA; #10;
```

```
        // 重新启用并读回
        CS = 1; WE = 0; RE = 1;
        Addr = 12'h900; #10;
        #50;
    end
endmodule
```

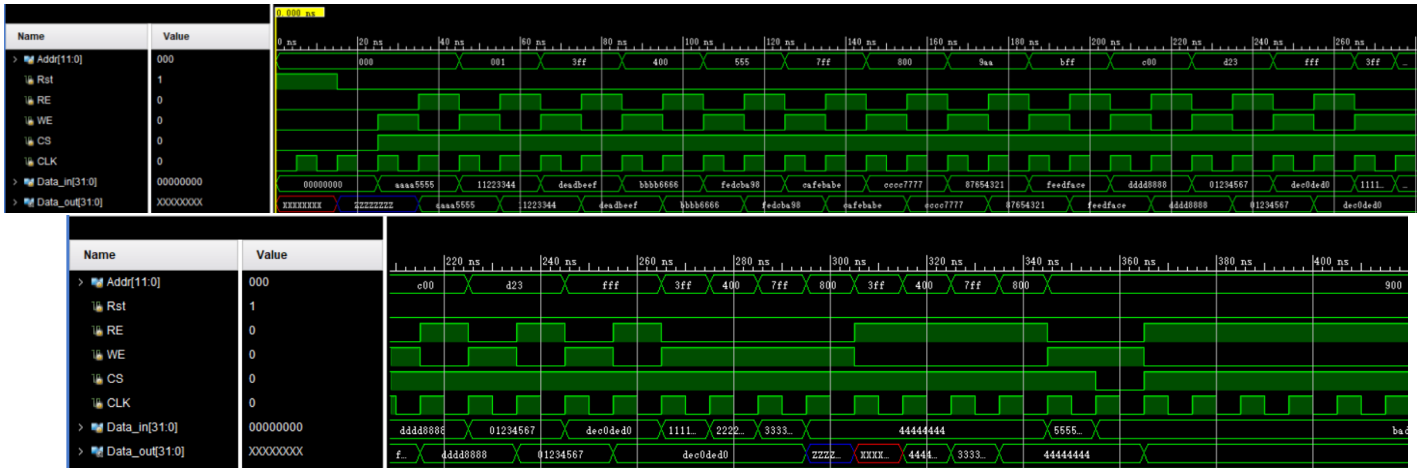
仿真策略：测试通过多组场景验证 4K×32 存储器的片选译码、读写功能及跨块访问：**测试 1：复位清零测试。**将 Rst 拉高一段时间后解除，清除整个 4K 存储空间的数据，确保初始时所有单元均为 0。

测试 2：分块读写测试。分别访问 4 个片选存储块 CS0~CS3 的典型地址，验证每个 1K×32 子存储块的独立读写功能。仿真中针对每个块执行写后读操作，例如：在 CS0 块地址 0x000 写入 32'hAAAA5555 后立即读取验证，在地址 0x001 写入 32'h11223344 后读取验证，并测试该块末地址 0x3FF 写入 32'hDEADBEEF；对 CS1、CS2、CS3 块也分别选取首地址、中间地址和末地址写入不同数据后读出比较。通过检查读出数据与写入值相符，确认每个片选块的读写逻辑正确无误，且各块之间的数据互不干扰。

测试 3：跨块地址连续访问。验证地址跨越存储块边界时的数据存取正确性。仿真先在 CS＝1、WE＝1 情况下，连续向相邻块交界处的地址写入数据：如地址 0x3FF（CS0 最后一地址）写入 32'h11111111，紧接着地址 0x400（CS1 起始地址）写入 32'h22222222，随后 0x7FF 写入 32'h33333333，0x800（CS2 起始地址）写入 32'h44444444。然后切换到读模式（WE＝0，RE＝1），依次读取上述地址。预期每个地址都读出各自对应的写入值，说明当地址从一块末端跳到下一块起始时，译码能正确切换存储单元，数据保持一致连续。此测试验证了地址高位译码分块机制在边界交接处的正确性。

测试 4：片选信号功能测试。验证顶层 CS 对整个存储器读写的控制作用。仿真中先使 CS＝1，对某地址写入数据 32'h55555555，然后将 CS 置 0 禁用存储器，再尝试向同一地址写入不同数据 32'hBADBADBA。由于 CS 已无效，第二次写操作应被忽略，不会更改存储内容。接着重新拉高 CS＝1，读取该地址。如果读出值仍为第一次写入的 32'h55555555，则证明 CS＝0 时写入被成功禁止，存储器保持了先前的数据不变。通过该测试确认全局片选信号有效地控制存储器的使能状态。

3、SIMULATION：



分析：复位测试首先在 0ns 将 Rst 拉高，保持到 15ns 后拉低释放，期间 4K×32 存储器的所有单元被清零。随后从 25ns 开始对各片选存储块进行独立读写测试：CS0 块（地址范围 0x000–0x3FF）片选有效时，依次在地址 0x000 写入 0xAAAA5555、地址 0x001 写入 0x11223344、以及最后一个地址 0x3FF 写入 0xDEADBEEF。每次写入后在 10ns 后对同一地址执行读操作，观察到 Data_out 分别输出与写入数据相同的 32 位值，验证该块存储正常。CS1 块（0x400–0x7FF）同样测试了地址 0x400 (写入 0xBBBB6666)、0x555 (写入 0xFEDCBA98) 和 0x7FF (写入 0xCAFEBABE) 的写入和读出；CS2 块（0x800–0xBFF）测试地址 0x800 (0xCCCC7777)、0x9AA (0x87654321) 和 0xBFF (0xFEEDFACE)；CS3 块（0xC00–0xFFF）测试地址 0xC00 (0xDDDD8888)、0xD23 (0x01234567) 和 0xFFF (0xDEC0DED0)。四个 1K×16 子模块的测试结果均表明各自地址的 Data_out 数据与写入值完全一致。接着进行跨存储块的连续地址读写实验：在 265ns 起依次对跨越 CS0~CS2 边界的地址执行写操作，每次间隔 10ns：写入 0x11111111 至地址 0x3FF（CS0 最后一地址）、0x22222222 至 0x400（CS1 起始地址）、0x33333333 至 0x7FF（CS1 最后一地址）、以及 0x44444444 至 0x800（CS2 起始地址）。然后从 305ns 开始切换为读模式，按相同地址顺序读取验证：315ns 读 0x3FF 得到 0x11111111，325ns 读 0x400 得到 0x22222222，335ns 读 0x7FF 得到 0x33333333，345ns 读 0x800 得到 0x44444444——可以看到跨片选边界的连续地址数据也均被正确存取。最后验证片选 CS 信号：345ns 时保持 CS=1 对地址 0x900 (属 CS2 范围) 写入 0x55555555；随后在 355ns 将 CS 拉低为 0 并再次对 0x900 发起写入 0xBADBADBA，由于芯片被禁用，此次写入没有生效；365ns 将 CS 恢复为 1、置 WE=0 和 RE=1 后读取地址 0x900，发现 Data_out 输出仍是之前写入的 0x55555555，证明 CS=0 时对存储器的读写操作被成功屏蔽。至此，RAM_4Kx32 模块的复位清零、分块寻址读写以及片选控制等功能均通过仿真波形得到了验证。

（三）选做：单数据线存储器设计（inout 类型接口）：

1、1K×16bit 存储器：

（1）代码：

```
module RAM_1Kx16_inout(Data, Addr, Rst, R_W, CS, CLK);

    parameter Addr_Width = 10;          //参数化地址线宽
    parameter Data_Width = 16;          //参数化数据线宽
    parameter SIZE = 2 ** Addr_Width;   //参数化大小 1024

    inout [Data_Width-1:0] Data;          //数据，输入输出类型
    input [Addr_Width-1:0] Addr;          //地址
    input Rst;                           //复位信号
    input R_W;                           //1 读_0 写信号
    input CS;                            //使能/片选信号
```

```
    input CLK;                              //时钟信号

    integer i;                             //临时变量,用于 for 循环
    reg [Data_Width-1:0] Data_i;

    reg [Data_Width-1:0] RAM [SIZE-1:0];    //1024*16 的 RAM

    //inout 类型接口输入输出控制
    assign Data = (R_W) ? Data_i : 16'bz;

    //always @(posedge CLK) begin   //时钟同步
    always @(*) begin              //异步
        casex({CS, Rst, R_W})
            4'bx1x :  for(i = 0; i <= SIZE-1; i = i+1)  RAM[i] = 0;
            4'b101 :  Data_i <= RAM[Addr];         //读数据
            4'b100 :  RAM[Addr] <= Data;           //写数据
            default : Data_i = 16'bz;
        endcase
    end
endmodule
```

分析：RAM_1Kx16_inout 模块在结构上与双数据线版本类似，内部使用参数化的 1024×16 位 RAM 数组存储数据，不同之处在于数据总线采用单一的双向端口 Data，并通过信号 R_W 区分当前的读/写模式。模块使用内部寄存器 Data_i 作为数据缓冲，通过三态门控制 Data 线的方向：使用连续赋值语句实现 assign Data = (R_W) ? Data_i : 16'bz;，当 R_W＝1（读模式）时，内部 Data_i 输出的数据驱动到 Data 总线；当 R_W＝0（写模式）时，模块将 Data 端口设为高阻态，由外部驱动写入数据。控制逻辑采用组合电路描述：always @(*) 根据 {CS, Rst, R_W} 的状态选择执行操作。当复位 Rst＝1 时（无论 CS 为何），通过 for 循环将 RAM 数组所有单元清零。若 CS＝1 且未处于复位状态，则依据 R_W 决定读写：R_W＝0 时执行写操作，模块从 Data 总线上采样输入数据写入地址 Addr 指定的内存单元；R_W＝1 时执行读操作，从内存中取出 Addr 地址的数据赋给内部 Data_i，并通过前述三态缓冲驱动到 Data 总线输出。默认情况下（如 CS＝0 或无有效读写请求时），模块将 Data_i 置为高阻，从而不会在总线上输出无效数据。通过以上机制，RAM_1Kx16_inout 利用单条数据线实现了存储器的数据双向读写，既保证总线互斥使用，又完整实现了复位、读、写等功能。

（2）仿真文件：

```
module sim_RAM_1Kx16_inout;
    reg CLK;
    reg CS;
    reg R_W;
    reg Rst;
    reg [9:0] Addr;
    wire [15:0] Data;

    reg [15:0] Data_driver;
    assign Data = (R_W==0) ? Data_driver : 16'bz;

    RAM_1Kx16_inout uut(
        .Data(Data),
        .Addr(Addr),
        .Rst(Rst),
        .R_W(R_W),
        .CS(CS),
        .CLK(CLK)
    );

    // 产生时钟
    initial begin
        CLK = 0;
        forever #10 CLK = ~CLK;
    end

    initial begin
        // 初始化信号
        CS  = 1;
        R_W = 1;     // 默认读
        Rst = 1;     // 复位
        Addr = 0;
        Data_driver = 16'h0000;

        #20 Rst = 0;
```

```
        //  写入阶段
        #20;
        R_W = 0;                // 写模式
        Addr = 10'd3;
        Data_driver = 16'hA5A5;

        #20;
        Addr = 10'd7;
        Data_driver = 16'h1234;

        //  读出阶段
        #20;
        R_W = 1;                // 读模式
        Addr = 10'd3;

        #30;
        Addr = 10'd7;

        #40;
        $finish;
    end

endmodule
```

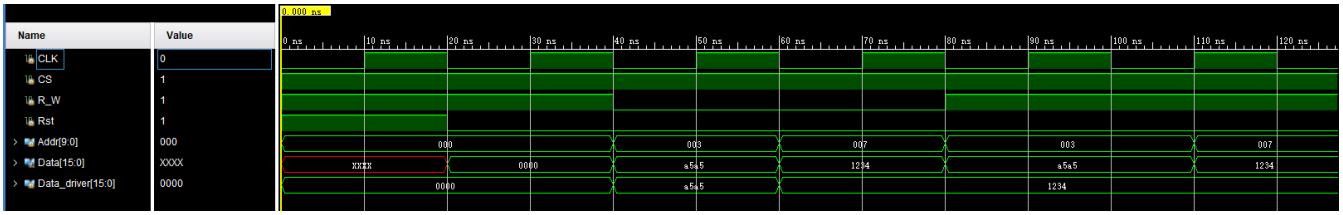
仿真策略：仿真首先对模块进行初始化和复位，然后分为写入和读出两个阶段验证单数据线读写的正确性：

①初始化复位：仿真开始时设置 CS＝1 保持芯片选中，R_W＝1 默认处于读模式，Rst 拉高进行复位初始化（清零存储器），随后在 20ns 后释放复位信号，确保存储器起始为空。

②写入阶段： 将 R_W 切换为 0（写模式），由测试平台通过附加的驱动信号向 Data 总线提供待写数据。首先设置 Addr＝3，Data 总线输入 16’hA5A5，保持一个时钟周期以写入该数据；接着设置 Addr＝7，Data 输入 16’h1234，再次等待一段时间完成写入。两次写操作后，预期地址 3 和 7 单元中分别存储了 16’hA5A5 和 16’h1234。

③读出阶段： 将 R_W 重新置为 1（读模式），然后提供先前写入的地址以从存储器读取数据。仿真依次将 Addr 设为 3 和 7，观察 Data 总线上输出的数据值：当 Addr＝3 时，应读出 16’hA5A5；Addr＝7 时，应读出 16’h1234。读出的数据与写入值一致，表明单数据线模式下存储器的写入和读出功能正常工作。整个过程中 CS 信号始终为 1 确保芯片处于选通状态，而复位操作保证了在进行读写测试前存储阵列处于已知的全零状态。

（3）SIMULATION：



分析：仿真开始时将 CS 置 1 使能芯片，Rst 拉高并维持 0-20ns 以清零 1K×16 RAM 的全部单元。20ns 时复位结束。随后 40ns 进入写入阶段：将 R_W 拉低为 0 切换到写模式，此时存储器通过 assign 将 Data 总线置为高阻，由外部测试平台的 Data_driver 控制总线电平。在 40ns 时，Addr 设为 0x003，Data_driver 在 Data 总线上提供数据 0xA5A5，该数据被立即写入地址 3 单元。60ns 时，Addr 改为 0x007，Data_driver 输出 0x1234，存储器将其写入地址 7。写入完成后，在 80ns 将 R_W 重新置为 1 切换回读模式，此时存储器开始驱动 Data 总线输出数据。首先保持 Addr=0x003 不变，存储器把地址 3 中存放的数值 0xA5A5 经 Data_i 加载后驱动到 Data 总线。在 80-110ns 时间内可以在波形上稳定观测到 Data 总线为 0xA5A5。紧接着在 110ns 将 Addr 切换为 0x007，Data 总线的输出随之更新为 0x1234，与之前写入该地址的数据一致。由此验证 RAM_1Kx16_inout 模块在单数据线模式下读写功能正常：写入时存储器释放数据总线由外部驱动数据，读出时存储器接管总线驱动输出正确的数据。

2、用 1K×16 位芯片构成 4K×32 位的存储器：

（1）代码：

```
module  RAM_4Kx32_inout          //4K×32 位 RAM
    #(parameter   Addr_Width = 12,    //参数化地址线宽
                  Data_Width = 32 )    //参数化数据线宽
    (inout   [Data_Width-1:0] Data,   //数据输入输出
    input   [Addr_Width-1:0] Addr,   //地址数据
    input   Rst,                     //复位信号
    input   R_W,                     //读写信号
    input   CS,                      //使能/片选信号
    input   CLK);                    //时钟信号

    wire   [3:0] CS_i;                //片选信号组
    Decoder24 Decoder24_1(CS_i, Addr[Addr_Width-1:Addr_Width-2]); //生成片选信号

    //字扩展，位扩展
                                                          RAM_1Kx16_inout
CS0_H_16bit(Data[Data_Width-1:Data_Width/2],  Addr[Addr_Width-3:0],  Rst
```

```
, R_W, CS_i[0], CLK),
                                    CS0_L_16bit(Data[Data_Width/2-1:0],
Addr[Addr_Width-3:0], Rst, R_W, CS_i[0], CLK);

                                                        RAM_1Kx16_inout
CS1_H_16bit(Data[Data_Width-1:Data_Width/2], Addr[Addr_Width-3:0], Rst
, R_W, CS_i[1], CLK),
                                    CS1_L_16bit(Data[Data_Width/2-1:0],
Addr[Addr_Width-3:0], Rst, R_W, CS_i[1], CLK);

                                                        RAM_1Kx16_inout
CS2_H_16bit(Data[Data_Width-1:Data_Width/2], Addr[Addr_Width-3:0], Rst
, R_W, CS_i[2], CLK),
                                    CS2_L_16bit(Data[Data_Width/2-1:0],
Addr[Addr_Width-3:0], Rst, R_W, CS_i[2], CLK);

                                                        RAM_1Kx16_inout
CS3_H_16bit(Data[Data_Width-1:Data_Width/2], Addr[Addr_Width-3:0], Rst
, R_W, CS_i[3], CLK),
                                    CS3_L_16bit(Data[Data_Width/2-1:0],
Addr[Addr_Width-3:0], Rst, R_W, CS_i[3], CLK);

endmodule
```

分析：RAM_4Kx32_inout 模块通过组合多个 RAM_1Kx16_inout 子模块，实现了 4K×32 位的单总线存储器架构。其结构与双数据线版本的 4K×32 类似：顶层设有全局片选信号 CS 用于整体使能存储器阵列，并使用地址高 2 位经 Decoder24 译码生成 4 个片选信号 CS_i[3:0]，将 4096 个地址划分为 4 个独立的 1K 地址块。每个地址块由两片 1K×16bit 单总线存储器组成，分别存储 32 位数据的高 16 位和低 16 位。地址译码产生的 CS_i 信号决定哪一块被选中，选中的块其两个子模块均接收相同的低 10 位地址 Addr[9:0]、共用的 Rst 复位和 R_W 读/写控制信号以及时钟 CLK。顶层 32 位 Data 总线按位段直接连接到所有子模块的数据端：其中 Data[31:16] 总线连接每组块内高位存储器，Data[15:0] 总线连接低位存储器。**读操作**时，当某块 CS_i＝1 且 R_W＝1 时，该块两片 16 位存储器会各自驱动 Data 总线相应的高半部和低半部输出数据；同时其他未被选中的块因 CS＝0 保持其 Data 端口高阻态，故总线上只有选中块的有效 32 位数据。**写操作**时，当 CS_i＝1 且 R_W＝0 时，表示选中了某块进行写入，外部驱动的 32 位总线数据将分别存储到该块的高位和低位子存储器中；其他未选块由于 CS＝0 不响应写请求，保持其总线端口不作用。通过这种方式，实现了地址空间和数据位宽的双扩展：利用地址高位片选实现纵向扩展存储容量，利用数据线分段实现横向扩展总线宽度，并确保任一时刻只

有目标片选块与总线通信，避免冲突。从层次上看，RAM_4Kx32_inout 顶层模块负责地址译码和片选控制，将读写请求路由到正确的子存储器，并通过共享总线完成数据的收发，从而在统一的地址空间下完成存储操作。

（2）仿真文件：

```
module sim_RAM_4Kx32_inout;

    reg CLK;
    reg CS;
    reg R_W;
    reg Rst;
    reg [11:0] Addr;

    wire [31:0] Data;

    // inout 驱动
    reg  [31:0] Data_driver;
    assign Data = (R_W==0) ? Data_driver : 32'bz;

    // 实例化 4K×32 位存储器
    RAM_4Kx32_inout uut(
        .Data(Data),
        .Addr(Addr),
        .Rst(Rst),
        .R_W(R_W),
        .CS(CS),
        .CLK(CLK)
    );

    initial begin
        CLK = 0;
        forever #10 CLK = ~CLK;
    end

    initial begin
        CS  = 1;
        R_W = 1;
        Rst = 1;
```

```
        Addr = 0;
        Data_driver = 32'h00000000;
        #20 Rst = 0;

        // 写 CS0（地址 0x000）
        #20;
        R_W = 0;
        Addr = 12'h000;          // bank 0
        Data_driver = 32'hA5A51234;

        // 写 CS1（地址 0x400）
        #20;
        Addr = 12'h400;          // bank 1
        Data_driver = 32'hFACE1234;

        // 写 CS2（地址 0x800）
        #20;
        Addr = 12'h800;          // bank 2
        Data_driver = 32'hDEADBEEF;

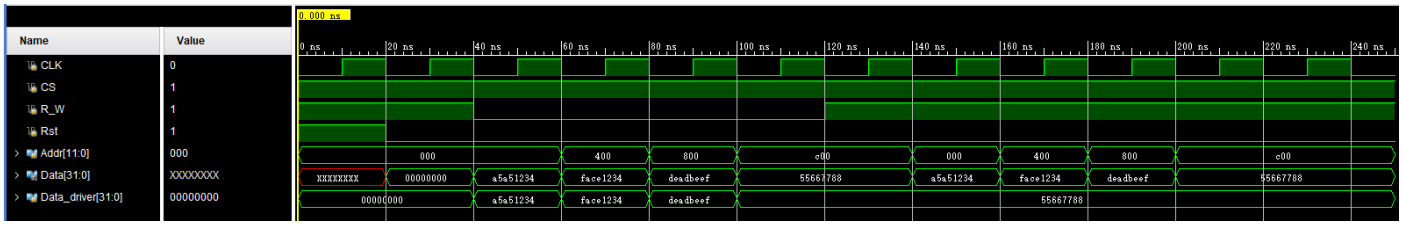
        // 写 CS3（地址 0xC00）
        #20;
        Addr = 12'hC00;          // bank 3
        Data_driver = 32'h55667788;

        // 读取测试
        #20 R_W = 1;  // 进入读模式
        // 读 CS0
        #20 Addr = 12'h000;
        // 读 CS1
        #20 Addr = 12'h400;
        // 读 CS2
        #20 Addr = 12'h800;
        // 读 CS3
        #20 Addr = 12'hC00;
        #50 $stop;
    end
endmodule
```

仿真策略：仿真通过对每个片选块进行一次写入和读出，验证多片 inout 存储器组合后的功能正确性：

首先进行复位，将 Rst 置 1 清除所有存储单元后解除，使存储器处于初始全零状态。接下来写入测试：保持 CS＝1 选通，总线处于写模式 (R_W＝0)，分别向四个地址区域的起始地址写入不同的 32 位数据模式。例如依次写入：Addr＝0x000 (CS0 区) 写 32’hA5A51234，Addr＝0x400 (CS1 区) 写 32’hFACE1234，Addr＝0x800 (CS2 区) 写 32’hDEADBEEF，Addr＝0xC00 (CS3 区) 写 32’h55667788。每次写入后暂停一小段时间，确保该操作完成后再进行下一操作。然后切换到读取测试：将 R_W 设为 1 进入读模式，保持 CS 有效，按照相同的地址顺序读出 0x000、0x400、0x800、0xC00 四个地址的内容。观察 Data 总线返回的数据应分别为 32’hA5A51234、32’hFACE1234、32’hDEADBEEF 和 32’h55667788，正好对应之前写入的值。由此验证：在单总线架构下，各片选块能够正确地存储和输出数据，且共享总线的读写控制与译码选择协同工作正常。整个过程中，全局 CS 信号维持为 1 保持器件工作，使能，Rst 确保了测试开始时存储器内容可控。通过上述操作，证明组合后的 4K×32_inout 存储器满足设计要求。

（3）SIMULATION：



分析：上电后首先在 0–20ns 间拉高 Rst 复位存储器，20ns 时释放复位。仿真随后演示跨 4 个片选存储块的写入和读出过程。在 40ns，将 R_W 置 0 进入写模式，依次对四个片选区域的基地址写入数据：40ns 写地址 0x000（CS0 起始地址），Data_driver 驱动 Data 总线送入 0xA5A51234，该 32 位数据被存入 CS0 块对应的存储单元；60ns 写地址 0x400（CS1 起始地址），写入数据 0xFACE1234；80ns 写地址 0x800（CS2 起始地址），写入 0xDEADBEEF；100ns 写地址 0xC00（CS3 起始地址），写入 0x55667788。完成各块写操作后，在 120ns 将 R_W 拉高为 1 切换到读出模式。随后从 140ns 起对上述四个地址逐一读取：Addr=0x000 时 Data 总线由存储器驱动输出 0xA5A51234；160ns 切换 Addr=0x400，Data 输出 0xFACE1234；180ns 地址改为 0x800，输出 0xDEADBEEF；200ns 地址 0xC00，输出 0x55667788。可见四个分片存储单元的数据均被正确读出，与先前写入值完全一致。整个过程中，单数据线 Data 的驱动方向会随 R_W 控制自动切换：写入时存储器释放总线由外部驱动数据，读出时存储器接管总线驱动数据输出，仿真波形显示的数值即为各地址实际存储的数据。

（四）指令存储器设计：

1、代码：

```
`define DATA_WIDTH 32

module IMem(//指令存储器
    input [5:0] A,
    output [`DATA_WIDTH-1:0] RD);

    parameter IMEM_SIZE = 64;

    //指令存储器
    reg [`DATA_WIDTH-1:0] RAM[IMEM_SIZE-1:0];

    initial begin
        RAM[0] = 32'h3C010001;
        RAM[1] = 32'h3424000C;
        RAM[2] = 32'h20050005;
        RAM[3] = 32'h20060007;
        RAM[4] = 32'h00A63820;
        RAM[5] = 32'hAC820000;
        RAM[6] = 32'h8C890000;
        RAM[7] = 32'h01284822;

        integer i;
        for(i = 8; i < IMEM_SIZE; i = i + 1)
            RAM[i] = 0;
    end

    assign RD = RAM[A];  // 指令输出
endmodule
```

分析：IMem 模块实现了一个 64×32 位的指令存储器，用于存放程序指令（只读存储）。地址输入 A 为 6 位，可寻址 64 个 32 位字。内部使用寄存器数组 RAM[0:63] 保存指令数据，并在模块初始化时 (initial begin 块) 对内容进行预加载：地址 0～7 预先存入了给定的指令机器码常量，地址 8～63 则通过循环被初始化为 0。由于该存储器仅用于读取指令，设计中没有写入端口，指令的输出 RD 由组合逻辑直接驱动：assign RD = RAM[A]，即根据地址 A 的值直接索引数组并输出相应的 32 位指令数据。这样当外部提供一个地址时，模块会立即输出对应单元的内容而无需额外控制信号。IMem 模块无时钟参与，属于异步只读存储结构；在实际处理器中，IMem 往往作为取指存储器，依靠预先烧录或初

始化的指令内容供 CPU 读取。该模块结构简单，信号接口明确：只需提供地址 A 即可得到对应的指令 RD。

2、仿真文件：

```
`define DATA_WIDTH 32

module sim_IMem;

    reg  [5:0] A;
    wire [`DATA_WIDTH-1:0] RD;

    // 实例化指令存储器
    IMem uut(
        .A(A),
        .RD(RD)
    );

    initial begin
        // 初始地址
        A = 0;

        // 观察前若干条指令
        #10 A = 6'd0;
        #10 A = 6'd1;
        #10 A = 6'd2;
        #10 A = 6'd3;
        #10 A = 6'd4;
        #10 A = 6'd5;
        #10 A = 6'd6;
        #10 A = 6'd7;
        #10 A = 6'd10;
        #10 A = 6'd20;

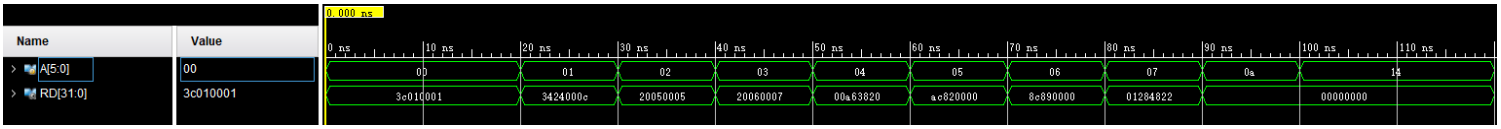
        #20 $stop;
    end

endmodule
```

仿真策略：仿真通过改变地址信号，顺序读取 IMem 中预先存储的指令来验证模块功

能。首先将地址 A 初始化为 0，然后每经过若干时间依次将 A 设置为 0、1、2…7，观察输出 RD 是否给出对应的指令码值。预期仿真波形中：当 A＝0 时 RD 输出 32’h3C010001，A＝1 时输出 32’h3424000C，A＝2 时输出 32’h20050005，…，直到 A＝7 时输出 32’h01284822，这些值应与 IMem 初始块中加载的指令机器码一致。接着仿真将地址跳变到 A＝10 和 A＝20 等超出预设指令范围的值，此时 RD 输出应为 0（因为这些地址在初始时被填充为 0）。通过以上测试，验证 IMem 模块能根据输入地址正确输出预存的指令内容：既能够读取有效的指令码，也会在地址超出已存范围时返回默认值 0，符合指令存储器的设计预期。

3、SIMULATION：



分析：IMem 模块内预置了 8 条 32 位指令（存放于地址 0-7），其余地址空间默认填充 0。仿真中无时钟，通过改变地址输入 A 来观察只读存储器的数据输出 RD。0ns 时 A=0，因此 RD 输出地址 0 处的指令机器码 0x3C010001。随后每隔 10ns 地址增加 1 以连续读取指令：10ns 时 A 仍为 0，RD 保持 0x3C010001；20ns 时 A=1，RD 更新为地址 1 的指令 0x3424000C；30ns 时 A=2，RD 输出 0x20050005；依次类推，80ns 时 A=7，RD 输出第 8 条指令 0x01284822。接着仿真跳转到更高地址进行测试：90ns 将 A 设为 10，此地址未被预置指令，因而 RD 输出 0x00000000（默认为 0）；100ns 时 A=20，RD 同样输出 0x00000000。由波形可见，IMem 模块能根据地址正确输出预存的指令数据，对于未初始化的地址则返回 0，符合设计预期。

（五）寄存器文件设计：

1、代码：

```
`define DATA_WIDTH 32

module RegFile //寄存器文件
    #(parameter ADDR_SIZE = 5)
    (input CLK, WE3,
     input [ADDR_SIZE-1:0] RA1, RA2, WA3,
     input [`DATA_WIDTH-1:0] WD3,
     output [`DATA_WIDTH-1:0] RD1, RD2);

    //寄存器文件
    reg [`DATA_WIDTH-1:0] rf[2 ** ADDR_SIZE-1:0];

    //数据写入
    always@(posedge CLK)
```

```
        if (WE3) rf[WA3] <= WD3;

    //数据读出
    assign RD1 = (RA1 != 0) ? rf[RA1] : 0;
    assign RD2 = (RA2 != 0) ? rf[RA2] : 0;

endmodule
```

分析：RegFile 模块实现了处理器中的寄存器文件，默认包含 32 个 32 位寄存器。模块提供一个写端口和两个独立读端口，以支持处理器同时读取两个源操作数的需求。内部使用一个长度为 2^5＝32 的寄存器数组 rf[0:31] 保存数据。写入路径采用时序逻辑控制：在时钟 CLK 上升沿触发时，如果写使能 WE3 为高，则将写数据 WD3 存入地址为 WA3 的寄存器单元，实现对该寄存器的新赋值；若 WE3 为低则不上升沿不执行写操作（寄存器内容保持不变）。读出路径为异步组合逻辑：输出 RD1 和 RD2 分别通过连续赋值语句直接连接到 rf 数组的读取端口 (rf[RA1] 和 rf[RA2])。为了满足 CPU 架构中寄存器 $zero 恒为 0 的约定，设计中对地址为 0 的读口作了特殊处理：当 RA1＝0 时强制 RD1 输出 0，而不是 rf[0] 存储的值；RA2 类似，对应地址为 0 时 RD2 输出 0。这样无论程序是否曾尝试写入寄存器 0，读出值始终为 0。该寄存器文件模块支持每个时钟周期写入一个寄存器，以及无延迟地同时读取两个寄存器，保证了处理器对寄存器的并行访问需求。其中对 $zero 寄存器的固定输出逻辑确保了寄存器 0 恒定为 0 的特性，实现方式是仅在读出时覆盖其输出，而没有在写入时特殊限制（因此 rf[0] 可能被写但读永远返回 0）。总体而言，RegFile 模块结构清晰：通过简单的时钟同步写和异步读机制，实现了多端口寄存器组的功能。

2、仿真文件：

```
`define DATA_WIDTH 32

module sim_RegFile;

    reg CLK;
    reg WE3;
    reg [4:0] RA1, RA2, WA3;
    reg [`DATA_WIDTH-1:0] WD3;
    wire [`DATA_WIDTH-1:0] RD1, RD2;

    // 实例化寄存器堆
    RegFile uut(
        .CLK(CLK),
        .WE3(WE3),
```

```
        .RA1(RA1),
        .RA2(RA2),
        .WA3(WA3),
        .WD3(WD3),
        .RD1(RD1),
        .RD2(RD2)
    );

    initial begin
        CLK = 0;
        forever #10 CLK = ~CLK;
    end

    initial begin
        WE3 = 0;
        RA1 = 0;
        RA2 = 0;
        WA3 = 0;
        WD3 = 0;

        // 写寄存器测试
        #10 WE3 = 1; WA3 = 5'd1; WD3 = 32'hAAAAAAAA;
        #10 WE3 = 1; WA3 = 5'd2; WD3 = 32'h12345678;
        #10 WE3 = 1; WA3 = 5'd3; WD3 = 32'hDEADBEEF;

        // 读寄存器测试
        #10 WE3 = 0; RA1 = 5'd1; RA2 = 5'd2;   // RD1=AAAAAAAA, RD2=12345678
        #10 RA1 = 5'd3; RA2 = 5'd0;            // RD1=DEADBEEF, RD2=0

        // 修改寄存器的值验证写后有效
        #10 WE3 = 1; WA3 = 5'd2; WD3 = 32'hFFFFFFFF;
        #10 WE3 = 0; RA1 = 5'd2;               // RD1=FFFFFFFF

        #30 $stop;
    end

endmodule
```

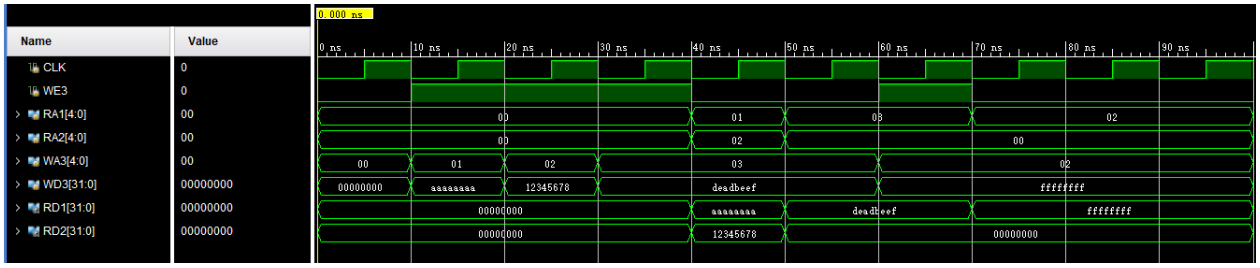
仿真策略：仿真采用分阶段的方法测试 RegFile 的写入、读取以及特殊寄存器行为：

首先是写寄存器阶段：在若干个连续的时钟上升沿，启用 WE3（=1）并提供不同的 WA3 和 WD3，对多个寄存器执行写操作。仿真中例如在三个周期内依次写入寄存器 1、2、3，写入的数据分别为 32'hAAAAAAAA、32'h12345678、32'hDEADBEEF。每次写入发生在时钟跳变瞬间，对应的寄存器地址和数据在上升沿前已经稳定有效，从而在该时刻将新值存入指定寄存器单元。

接下来进行读寄存器阶段：将 WE3 置 0 停止写操作，然后设置 RA1、RA2 为特定值以读取之前写入的寄存器内容。首先读取寄存器 1 和 2（如 RA1=1，RA2=2），此时输出的 RD1 应为 0xAAAAAAAA，RD2 为 0x12345678，与之前写入的数据一致；然后更改读地址为 RA1=3，RA2=0，读取寄存器 3 和 0。预期 RD1 输出 0xDEADBEEF（寄存器 3 的值），而 RD2 输出 0（寄存器 0 特性导致其读出恒为 0），验证了双端口读功能以及寄存器 0 恒零的约束。

最后进行写后覆盖验证：再次对已有值的寄存器执行写入以检验覆盖效果。例如仿真在后续时钟周期将寄存器 2 更新写入新值 32'hFFFFFFFF，然后马上将 WE3 置 0 并设置 RA1=2 读取该寄存器。结果 RD1 应读出 0xFFFFFFFF，不再是先前的 0x12345678，从而确认新的写操作成功覆盖旧值，寄存器文件能够正确更新寄存器内容。整个仿真过程中时钟 CLK 持续运行以驱动写入发生，读操作因为是组合逻辑读出则可以在非时钟边沿即时反映。通过上述一系列测试，验证了 RegFile 模块的基本功能：时序写入、多端口同时读出以及特殊寄存器的正确行为。

3、SIMULATION：



分析：寄存器文件包含 32 个 32 位寄存器。仿真波形展示对寄存器文件的写入和读出过程。初始状态下所有控制信号为 0，RA1=RA2=0，所以 RD1、RD2 均输出 0。时钟 CLK 从 0ns 开始每隔 10ns 翻转一次。在 10ns 第一个上升沿到来时，写使能 WE3=1，写地址 WA3=1，写数据 WD3=0xAAAAAAAA，因此此时寄存器 1 被写入 0xAAAAAAAA。20ns 时第二个时钟上升沿到来，WA3 已更新为 2、WD3=0x12345678 (WE3 保持为 1)，寄存器 2 被写入 0x12345678；30ns 第三个上升沿时，WA3=3、WD3=0xDEADBEEF，寄存器 3 写入 0xDEADBEEF。完成上述写入后，40ns 起将 WE3 清 0 禁止后续写入，并设置 RA1=1、RA2=2 以读取寄存器 1 和 2：此时 RD1 输出 0xAAAAAAAA，RD2 输出 0x12345678，与先前写入的数据吻合。50ns 将 RA1 切换为 3、RA2 切换为 0，此时 RD1 读出寄存器 3 的值 0xDEADBEEF，而 RD2 读出寄存器 0 (固定为 0)，因此 RD2 输出 0。随后在 60ns 再次开启写使能 WE3=1，设置 WA3=2、WD3=0xFFFFFFFF，准备改写寄存器 2；70ns 时钟上升沿触发寄存器 2 更新为 0xFFFFFFFF。紧接着在 70ns 将 WE3 拉低为 0 并将 RA1 指向 2，开始读出寄存器 2，观察到此时 RD1 输出 0xFFFFFFFF，成功读取到寄存器 2 的新值。整

个仿真过程中，寄存器文件的写操作仅在时钟上升沿触发更新，读操作则为组合逻辑即时输出，波形显示结果与设计预期完全一致。

五、调试和心得体会

1、问题与调试：

①仿真数据无法正确输出或波形不变化：初期我在读写控制信号组合逻辑上理解不够清晰，导致部分测试中读写时序不一致，或者 RE/WE 设置不当（例如同时置 1），使得 Data_out 信号为高阻态 (z)，无法在仿真波形中观察到有效数据。通过细致分析模块内部 casex 控制逻辑以及波形时序，我明确了控制信号组合的合法性，并逐步修正测试代码。

②片选信号未正确工作：在 4K×32 模块中，地址译码使用高位生成片选信号，由于我一开始未正确连接 Addr 高位到 Decoder 模块，导致访问某些地址时没有激活目标片选块。通过在仿真中观察 CS_i 信号的变化情况，我确认了片选逻辑的问题，并通过正确划分地址位段（Addr[11:10]）解决了此问题。

③inout 接口信号冲突问题：在实现 RAM_1Kx16_inout 模块时，由于 Data 为双向端口，需要正确控制何时由模块驱动，何时由测试平台驱动。我起初未设置三态控制，导致在读写切换时 Data 总线出现冲突或无输出。通过添加条件赋值语句 assign Data = (R_W) ? Data_i : 16'bz; 成功解决了这一问题。同时，仿真测试中也需要同步使用 assign Data = (R_W == 0) ? Data_driver : 16'bz; 控制数据流向，避免总线驱动冲突。

④模块复用和层级测试思路的理解：尤其是在 4K×32 的顶层模块中，如何组织多个子模块并通过统一接口控制其读写是一个挑战。我学会了使用片选信号统一调度不同块，以及通过对地址高位进行译码来管理多个 RAM 单元的分区读写逻辑，这对理解片选机制、总线互斥、模块扩展有重要帮助。

2、心得体会：

通过此次实验，我收获了如下心得体会：

①代码结构的重要性：模块化设计、参数化定义能极大提高代码的复用性与可读性，也便于进行不同配置存储器的扩展；

②仿真验证不可忽视：只有通过充分的仿真策略，验证读写正确性、边界情况、非法输入处理，才能真正保证设计的可靠性；

③控制信号时序需准确：理解时序电路与组合逻辑在 Verilog 中的区别，明确在何种条件下更新数据，避免误读误写；

④存储器的系统性认知：本实验使我掌握了从基本存储单元、片选译码到寄存器文件构造的整个过程，加深了对存储结构原理和应用的理解。

3、总结：

本次实验不仅提升了我使用 Verilog 和 Vivado 进行系统性设计和验证的能力，也让我在面对问题时学会了冷静分析和逐步排查。未来制作更复杂的处理器设计或存储子系统构建时，这些经验都将是宝贵的基础。